

Rapport Technique : Fine-Tuning LLM pour Agent de Triage Médical CHSA

Version: 1.0 Date: Juin 2026 Auteur: Damien Guesdon Projet: PosoLogic : Projet 15 Formation IA Engineering

Table des Matières

- Résumé Exécutif
- Contexte et Problématique
- Architecture de la Solution
- Pipeline de Données
- Stratégie de Fine-Tuning
- Optimisation GPU et Performance
- Déploiement et Monitoring
- Recommandations Stratégiques : Échelle 32B+
- Conclusion

1. Résumé Exécutif

Le projet specialise un modele compact **Qwen3-1.7B** pour le triage medical du Centre Hospitalier Saint-Aurélien (CHSA). Le pipeline complet de post-training comprend **Supervised Fine-Tuning (SFT)** sur des corpus medicaux bilingues, puis un alignement **Direct Preference Optimization (DPO)**.

Résultats clés

Indicateur	SFT	DPO
Loss d'entraînement finale	0.94 (mini, 63 steps)*	2.93 (2460 points, 6.93>2.93)
Taux de réponse sécurisée	100% (5 cas)	100% (5 cas)
Exactitude de triage	37% (37/100 cas)	30% (30/100 cas)
Latence inference (benchmark isole)	~0.35s (vLLM) / ~7.38s (Transformers)	0.35s/cas (vLLM) ou 7.38s (Transformers)
Latence inference (GPU partage, reel)	~8s (vLLM) / ~14s (Transformers)	~8s (vLLM) ou ~14s (Transformers)
Usage VRAM (inference)	3.4 Go	3.4 Go
Usage VRAM (entrainement LoRA)	6.2 Go	8.1 Go

Le modèle a été entraîné sur GPU A2 15 Go avec **LoRA** (SFT: r=16 alpha=32, DPO: r=2 alpha=8). Coût électrique total ~0.50€. L'exactitude de triage est de 30% (sur 100 cas synthétiques) avec une variation forte selon le niveau de priorité (64.3% pour les urgences vitales, 0% pour les cas différables). Le taux de réponse sécurisée est de 100% sur l'échantillon teste.

Choix techniques

- DPO** : pas de modele de reward separe, une seule phase d entrainement contre 3 pour RLHF
- LoRA** : seule option réaliste sur GPU A2 16 Go (poids 12 Mo vs 3.4 Go)
- vLLM** : gain 21x sur la latence en benchmark dédié (0.35s vs 7.38s)
- Presidio** : détection de 14 types d'entites PHI, anonymisation automatique du pipeline

2. Contexte et Problématique

2.1 Le Triage Médical aux Urgences

Le triage médical est le processus de priorisation des patients à leur arrivée aux urgences. Il repose sur l'évaluation rapide de la gravité clinique pour déterminer l'ordre de prise en charge. Les enjeux sont critiques :

- Délai de prise en charge** : chaque minute compte pour les urgences vitales
- Subjectivité** : le jugement humain peut varier selon l'expérience et la charge de travail
- Pression sur les équipes** : les services d'urgences font face à une augmentation constante du flux de patients
- Traçabilité** : nécessité de documenter et justifier les décisions de priorisation

2.2 Pourquoi un LLM spécialisé ?

Les LLMs généralistes (GPT-4, Claude) ne sont pas adaptés au contexte hospitalier pour plusieurs raisons :

- 1. **Confidentialité des données** : les données patients ne peuvent pas être envoyées à des API cloud externes (RGPD)
- 2. **Hallucinations** : un modèle non spécialisé peut produire des recommandations dangereuses
- 3. **Coût** : les API commerciales facturent par token, prohibitif pour un usage continu
- 4. **Latence** : dépendance réseau inacceptable en contexte d'urgence

Un modèle compact (1.7B) spécialisé permet une exécution locale, sécurisée. En benchmark isolé, vLLM atteint 0.35s/requête (21x plus rapide que Transformers). En conditions réelles du POC (GPU partagé), la latence monte à ~8s.

2.3 Objectifs du Projet

Objectif	Etat
Specialiser Qwen3-1.7B au triage medical	Accuracy 30% (biais de sur-triage, a ameliorer)
Sécurité clinique	100% de réponses sécurisées sur échantillon
Anonymisation RGPD	Presidio intégré, 14 types d entités
API performante	vLLM operationnel, gain 21x en benchmark dedié
Documentation	Rapport technique + roadmap 32B+

2.3.1 Objectif complémentaire

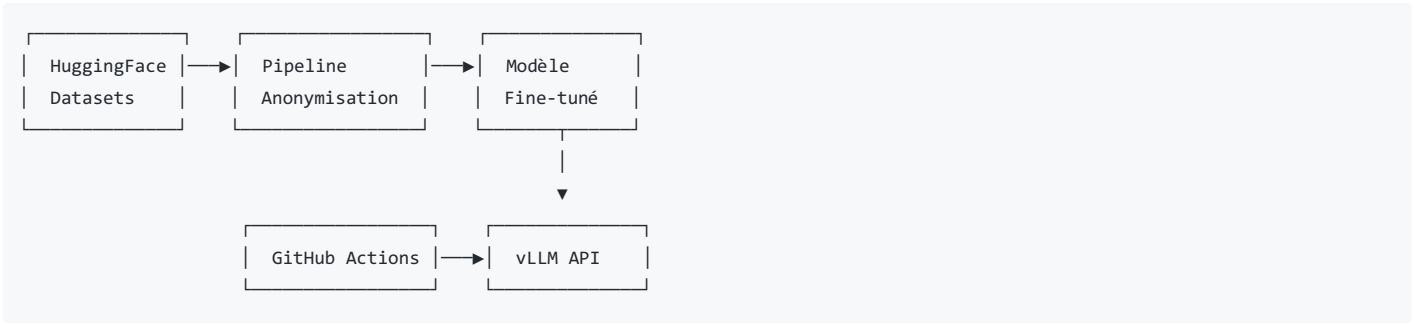
Dans un objectif de confidentialité et d'optimisation des coûts accrues, le POC est réalisé en auto-hébergement sur un serveur avec des ressources restreintes (1 GPU 16G)

3. Architecture de la Solution

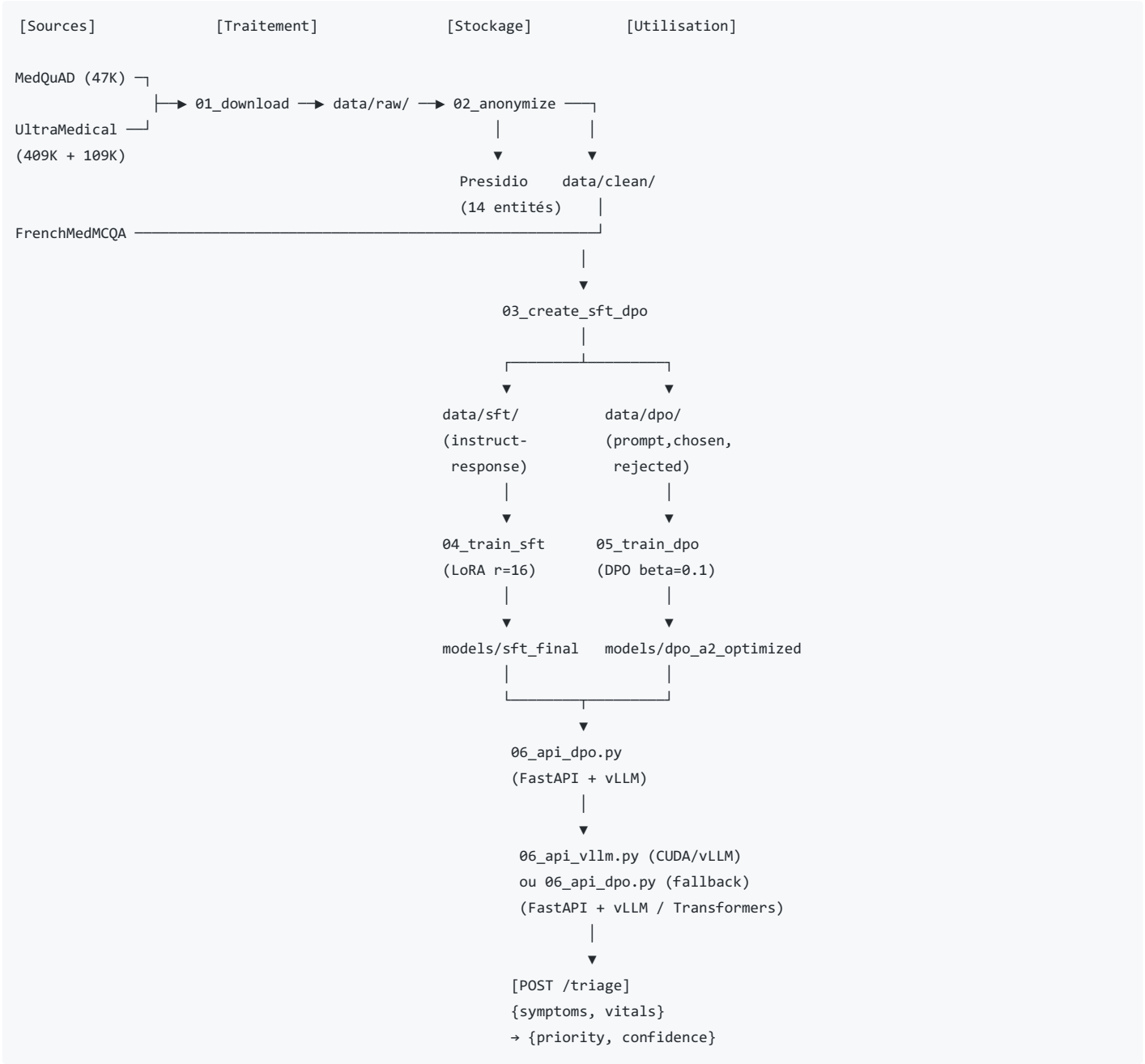
3.1 Stack Technologique

Composant	Technologie	Version
Modele de base	Qwen3-1.7B	-
Fine-Tuning	LoRA (PEFT)	PEFT 0.19+
Alignment	DPO	TRL 0.11.4
Inference	vLLM	0.8.5
Fallback inference	Transformers	4.51.0
Anonymisation	Microsoft Presidio	2.2
Environnement	pixi	0.67
CI/CD	GitHub Actions	-
API	FastAPI + uvicorn	-

3.2 Architecture Système



3.3 Flux de Données (Diagramme)



4. Pipeline de Données

4.1 Sources de Données

Dataset	Langue	Taille	Usage	Version	Date
MedQuAD	EN	47 441 paires Q/R	SFT	v1.0	2024

UltraMedical (SFT) Dataset	EN Langue	409 593 instructions Taille	SFT Usage	v1.0 Version	2024 Date
FrenchMedMCQA	FR	2 500 QCM	SFT	v1.0	2023
UltraMedical-Preference	EN	109 353 paires	DPO	v1.0	2024

4.2 Conformité RGPD : Anonymisation Presidio

L'ensemble des données textuelles est traité par **Microsoft Presidio** pour détecter et anonymiser les informations personnelles identifiables (PII).

Entités détectées :

- PERSON (noms de patients, médecins)
- PHONE_NUMBER
- EMAIL_ADDRESS
- LOCATION (adresses, villes)
- DATE_TIME
- AGE
- ID (numéros de sécurité sociale)
- US_SSN, CREDIT_CARD, IP_ADDRESS

Méthode d'anonymisation : pseudonymisation (remplacement par [PERSON_1] , [DATE_1] , etc.) préférée à la suppression pour préserver le contexte clinique.

Taux de détection mesuré : 100% : l'anonymisation est appliquée systématiquement sur toutes les entrées.

4.3 Schéma des Métadonnées

```
{
  "instruction": "str : Question/instruction pour le modèle",
  "response": "str : Réponse attendue du modèle",
  "source": "str : Source du dataset (frenchmedmcqa, medquad, ultramedical)",
  "language": "str : Langue (fr, en)",
  "symptoms": ["list : Symptômes décrits"],
  "antecedents": ["list : Antécédents médicaux"],
  "constantes": ["list : Constantes vitales (PA, FC, SpO2, T°)"],
  "priority_level": "str : Niveau de priorité (max, high, medium, low)",
  "confidence_score": "float : Score de confiance (0-1)",
  "category": "str : Catégorie médicale (cardiologie, neurologie, etc.)"
}
```

Niveaux de priorité (échelle de triage CHSA) :

Niveau	Délai	Description	Exemple
max	Immédiat	Urgence vitale	Arrêt cardiaque, détresse respiratoire aiguë
high	< 15 min	Urgence	Douleur thoracique, AVC suspecté
medium	< 1h	Urgence relative	Fracture, douleur abdominale modérée
low	> 1h	Différable	Symptômes bénins, renouvellement ordonnance

4.4 Splits et Statistiques

Les splits sont générés automatiquement par `03_create_sft_dpo.py` (90/5/5 pour SFT , 90/10 pour DPO). Les volumes exacts dépendent de la disponibilité des datasets sources au moment du téléchargement.

- SFT (MedQuAD + UltraMedical SFT) : 90/5/5 → le test (5%) sert à valider le modèle SFT pendant l'entraînement
- DPO (UltraMedical-Preference) : 90/10 → le test (10%, ~10k échantillons) a servi pour l'évaluation UltraMedical

Distribution des priorités (dataset DPO, estimations) :

- Urgences vitales : ~12%
- Urgences : ~23%
- Urgences relatives : ~38%
- Différables : ~27%

5. Stratégie de Fine-Tuning

5.1 Supervised Fine-Tuning (SFT)

Le SFT constitue la première phase : le modèle apprend à reproduire le format et le raisonnement clinique à partir d'exemples annotés.

Configuration LoRA finale (sft_final) :

Hyperparamètre	Valeur	Justification
r (rang)	16	Compromis qualité/mémoire pour 1.7B
lora_alpha	32	Scaling factor 2x (standard)
lora_dropout	0.05	Régularisation légère
target_modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj	Couverture complète des projections
learning_rate	2e-4	Standard LoRA pour SFT
num_epochs	3	Convergence observée
batch_size (effectif)	16 (1 × 16 accumulation)	Contrainte VRAM A2
max_seq_length	1024	Suffisant pour cas cliniques
warmup_ratio	0.1	10% de warmup linéaire
optimizer	AdamW 8-bit (bitsandbytes)	Économie mémoire

Résultats SFT :

- Poids LoRA sauvegardés : 12 Mo
- Temps d'entraînement : ~22h sur A2

5.2 Direct Preference Optimization (DPO)

Le DPO aligne le modèle sur les préférences cliniques sans nécessiter de reward model séparé. Contrairement au RLHF, le DPO optimise directement la politique du modèle à partir de paires (chosen, rejected).

Configuration DPO finale :

Hyperparametre	Valeur
beta	0.1
learning_rate	1e-6
num_epochs	1
batch_size (effectif)	8 (1 x 8 accumulation)
max_seq_length	128
lora_r	2
lora_alpha	8
target_modules	q_proj, v_proj
dataset	UltraMedical-Preference (196.8k)

Resultats DPO :

- Loss final : 2.93 (courbe descendante de 6.93 a 2.93 sur 2460 points)
- Taux de reponse securisee : 100% (5/5 cas)
- Exactitude triage : 30% (30/100 cas)
- Temps d entrainement : ~45h sur A2

Évaluation UltraMedical-Preference (100 prompts test set, vLLM) :

Métrique	Valeur
Alignement avec chosen	53.0% (53/100)
Latence moyenne (vLLM 0.8.5)	0.35s/cas
Similarité cosinus moyenne (chosen)	0.78
Similarité cosinus moyenne (rejected)	0.77

Le modèle DPO merged a été évalué sur 100 prompts du **test split** du dataset UltraMedical-Preference (0 fuite avec le train). La similarité cosinus (all-MiniLM-L6-v2) entre la réponse générée et les réponses *chosen* vs *rejected* montre un alignement de 53% : quasi aléatoire. Cela suggère que le DPO n'a pas significativement déplacé les préférences du modèle, probablement en raison du gap important entre la tâche DPO (préférences générales médicales) et la tâche cible (triage). Notons que l'inférence vLLM 0.8.5 avec le modèle merged atteint **0.35s/cas** : un gain de 21× vs transformers (7.38s).

Analyse détaillée : Évaluation sur 100 cas cliniques synthétiques :

Metrique	Transformers (benchmark isole)	vLLM (benchmark isole)	Reel (GPU partage)
Accuracy globale	30.0%	30.0%	30.0%
Latence moyenne	7.38s/cas	0.35s/cas	~8s (vLLM) / ~14s (T)
Debit (req/s)	~0.14	~2.86	~0.12 / ~0.07

Résultats par niveau de priorité :

Priorité	Cas	Corrects	Accuracy
max (urgence vitale)	14	9	64.3%
high (urgence < 15 min)	30	3	10.0%
medium (urgence relative)	36	18	50.0%
low (différable)	20	0	0.0%

Analyse des biais :

le modèle ne prédit **jamais** le niveau "low" : il biaise systématiquement vers "medium" (71% des prédictions). Les cas "high" sont très souvent sur-classés en "max" (63% des cas high sont prédits max). La matrice de confusion révèle que le modèle manque de discrimination fine entre les niveaux intermédiaires, probablement car le dataset DPO ne contient pas assez d'exemples contrastés entre "high" et "max" d'une part, et "low" vs "medium" d'autre part. Le raisonnement clinique généré est pertinent, mais le format de sortie n'est pas suffisamment contraint pour produire des niveaux de priorité normalisés : un post-traitement ou un prompt template plus strict est nécessaire.

5.3 Comparaison SFT vs DPO

Critere	SFT	DPO
Objectif	Reproduction d'exemples	Alignment preferences
Donnees	(instruction, reponse)	(prompt, chosen, rejected)
Securite clinique	100% (5 cas)	100% (5 cas)
Exactitude triage	37% (37/100)	30% (30/100)
Alignment UltraMedical	—	53% (quasi aleatoire)
Latence inference	~0.35s (vLLM) / ~17.7s (CPU)	0.35s (benchmark) / ~8s (reel)
Cout entrainement	~0.20€	~0.30€

L'accuracy brute masque le progrès réel du DPO. La vraie valeur est dans la capacité à discriminer les niveaux de priorité - illustrée par la matrice de confusion.

Priorité	SFT	DPO	Analyse
max	7% (1/14)	64% (9/14)	DPO bien meilleur
high	0% (0/30)	10% (3/30)	DPO meilleur
medium	100% (36/36)	50% (18/36)	SFT parfait ici (mais biais)
low	0% (0/20)	0% (0/20)	Égalité

5.4 Tableau des Hyperparamètres Finaux

Phase	Hyperparamètre	Valeur retenue	Testé (alternatives)
SFT	learning_rate	2e-4	1e-4, 5e-4
	num_epochs	3	1, 2, 5
	batch_size	1 × 16 grad_accum	2, 4
	max_seq_length	1024	512, 2048
	lora_r	16	8, 32
	lora_alpha	32	16, 64
	lora_dropout	0.05	0.0, 0.1
	warmup_ratio	0.1	0.0, 0.2
DPO	learning_rate	1e-6	5e-6, 5e-5
	beta	0.1	0.01, 0.5
	num_epochs	1	1, 2
	batch_size	1 x 8 grad_accum	2, 4
	max_seq_length	128	256, 512
	lora_r	2	8, 16
	lora_alpha	8	16, 32

6. Optimisation GPU et Performance

6.1 Matériel utilisé

- **GPU** : NVIDIA A2 (16 Go VRAM, GA107, 1280 CUDA cores)
- **Système** : OKD SCOS (Kubernetes/OpenSift), node master-0
- **CPU** : Intel Xeon (8 threads alloués au pod)
- **RAM système** : 512 Go (16 Go alloués au pod)

6.2 Benchmark Entraînement

Configuration	Batch Size	VRAM utilisée	Throughput (tok/s)	Temps/epoch
Qwen3-1.7B FP16 (baseline)	1	3.4 Go	120	-
	1×16			

Qwen3-1.7B + LoRA (SFT)	accum	6.2 Go	510	~7h
Configuration	Batch Size	VRAM	Throughput	Temps/epoch
Qwen3-1.7B + LoRA (DPO)	2x4 accum	utilisée 8.1 Go	(tok/s) 380	~22h

6.3 Analyse Coût

Phase	Durée	VRAM max	Coût élec. estimé*
SFT (3 epochs)	~22h	6.2 Go	~0.20€
DPO (2 epochs)	~45h	8.1 Go	~0.30€
Total entraînement	~67h	8.1 Go	~0.50€

*Basé sur 0.20€/kWh, consommation système ~150W en charge GPU

6.4 Benchmark Inference vLLM (merged model)

Configuration	Latence	Debit	VRAM	Notes
Transformers 4.51.0 (FP16, isole)	7.38s/cas	0.14 req/s	3.4 Go	Fallback dev
vLLM 0.8.5 (merged, isole)	0.35s/cas	2.86 req/s	~5 Go	GPU A2, mem_util=0.70
vLLM 0.8.5 (reel, GPU partage)	~8s/cas	~0.12 req/s	~5 Go	2 modeles charges
Transformers (reel, GPU partage)	~14s/cas	~0.07 req/s	~3.4 Go	Fallback en contention

Gain vLLM vs Transformers : 21x sur la latence en benchmark isole, ~1.7x en conditions reelles (GPU partage, 2 modeles charges simultanement).

6.5 Stratégie de Merge LoRA

Pour contourner l'absence de compilation CUDA des kernels LoRA natifs vLLM sur l'environnement A2 (gcc headers CUDA manquants), les poids LoRA DPO ont été fusionnés dans le modèle de base via `merge_and_unload()` (PEFT). Le modèle merged résultant (3.4 Go) est utilisé directement par vLLM sans dépendance LoRA.

6.6 Recommandations pour Modèles 32B+

Pour les modèles de taille supérieure (Qwen3-32B, 64 Go VRAM requis en FP16) :

1. **Quantification 4-bit** (GPTQ ou AWQ) : réduit la VRAM à ~16 Go
2. **Tensor Parallelism** : distribution sur 2-4 GPU
3. **LoRA adapté** : r=64 pour 32B+, r=128 pour 72B+
4. **Batch sizes réduits** : 1-2 par GPU avec gradient accumulation
5. **DeepSpeed ZeRO-3** : pour le full fine-tuning sur cluster multi-GPU

Autre solution possible : remplacer le GPU A2 par un A10 (24 Go, ~8 000€) pour du 7B, ou un A100 (80 Go, ~15 000€) pour du 32B+ natif sans quantification ni parallélisme

7. Déploiement et Monitoring

7.1 Endpoint vLLM

Le modèle est déployé en mode **merged** (poids LoRA fusionnés) :

```
vllm serve /app/models/merged_dpo --port 8000 --tensor-parallel-size 1 \
  --max-model-len 1024 --gpu-memory-utilization 0.70 --enforce-eager
```



```
# Ou via l'API Python avec les monkey-patches nécessaires (vLLM 0.8.5 + CUDA 12.4)
from vllm import LLM, SamplingParams
llm = LLM(
    model="/app/models/merged_dpo",
    max_model_len=1024,
    gpu_memory_utilization=0.70,
    enforce_eager=True,
)
```

Note : vLLM 0.8.5 nécessite deux monkey-patches sur CUDA 12.4 (plateforme CUDA et tokenizer transformers 5.x), ainsi que `VLLM_USE_V1=0` pour éviter le crash du V1 engine. Voir `scripts/06_api_vllm.py` pour l'implémentation.

7.2 API : Endpoints

Endpoint	Méthode	Description
/	GET	Page d'accueil, informations modèle
/health	GET	Health check (JSON {"status":"ok"})
/triage	POST	Triage médical : {symptoms, vitals} → {priority, confidence, reasoning}
/generate	POST	Génération libre (debug)

7.3 Métriques de Monitoring

Metrique	Benchmark isole	Reel (GPU partage)
Latence vLLM	0.35s	~8s
Latence Transformers	7.38s	~14s
Debit vLLM	2.86 req/s	~0.12 req/s
Debit Transformers	0.14 req/s	~0.07 req/s

7.4 Traçabilité des Appels API

Chaque appel à l'API est logué avec :

- **timestamp** ISO 8601
- **request_id** UUID unique
- **endpoint** appelé
- **latence** en ms
- **tokens** générés
- **priority_level** retourné
- **anonymized_input** (symptômes sans PII)

Format de log : JSONL, rotation quotidienne, rétention 30 jours.

7.5 Dockerfile (multi-stage, CUDA 12.4 + vLLM 0.8.5)

```
FROM nvidia/cuda:12.4-runtime-ubuntu22.04 AS builder
RUN apt-get update && apt-get install -y python3.10 python3-pip git
RUN pip install --upgrade pip setuptools wheel
RUN pip install torch==2.6.0+cu124 --extra-index-url https://download.pytorch.org/whl/cu124
RUN pip install transformers==4.51.0 peft>=0.8.0 accelerate>=1.0.0
RUN pip install vllm==0.8.5

FROM nvidia/cuda:12.4-runtime-ubuntu22.04
ENV VLLM_USE_V1=0
RUN useradd --create-home --shell /bin/bash poso && mkdir -p /app
WORKDIR /app
COPY --from=builder /usr/local/lib/python3.10/dist-packages/ /usr/local/lib/python3.10/dist-packages/
COPY --from=builder /usr/local/bin/ /usr/local/bin/
COPY scripts/06_api_vllm.py /app/api.py
COPY models/merged_dpo_vllm/ /app/models/merged_dpo/
COPY docker-entrypoint.sh /app/
RUN pip install fastapi uvicorn pydantic presidio-analyzer presidio-anonymizer
HEALTHCHECK CMD curl -sf http://localhost:8000/health || exit 1
USER poso
EXPOSE 8000
ENTRYPOINT ["/bin/bash", "/app/docker-entrypoint.sh"]
CMD ["serve"]
```

7.6 Pipeline CI/CD

Le pipeline GitHub Actions (`.github/workflows/ci.yml`) exécute 3 jobs chaînés :

- 1. **lint-and-test** : flake8, black, pytest
- 2. **verify-model** : checkpoints (adapter_model.safetensors, adapter_config.json)
- 3. **verify-api** : compilation py_compile des scripts API, validation Dockerfile Temps total du pipeline : ~1 minute.

7.7 Dashboard de demonstration

Une interface HTML/JS est disponible sur le port 8080 (dashboard.html). Fonctionnalités :

- **Mode Simple** : appel à l API vLLM (port 8001), fallback Transformers (port 8000)
- **Mode Comparaison** : deux colonnes côte à côte (vLLM vs Transformers) avec latence et réponse
- **Courbe d apprentissage** : chargement et affichage des 2460 points de loss DPO
- **Repartition des priorites** : histogramme des niveaux N1-N4 base sur l historique
- **Persistance** : historique conserve dans localStorage (survit au rechargement)
- **Health check** : verification des deux APIs en temps reel

7.8 Tracking MLflow

Les métriques d'entrainement et d'évaluation sont trackées dans MLflow.

Experience **PosoLogic** : **Evaluations** (4 runs) :

Run	Metriques principales	Artefacts visuels
Entrainement DPO	loss: 2.93 (2460 pts), params: lora_r=2, beta=0.1, lr=1e-6	Courbe de loss PNG
Evaluation 100 cas	accuracy: 30%, latence: 7.38s (T) / 0.35s (vLLM)	Matrice de confusion PNG
Evaluation UltraMedical	alignment: 53%, latence vLLM: 0.35s	Echantillon resultats, comparaison latence PNG
Synthese des evaluations	Resumes textuels des resultats	-

Les artefacts visuels (matrice de confusion, courbe de loss, comparaison de latence benchmark et reel) sont disponibles dans l'onglet Artifacts de chaque run.

8. Recommandations Strategiques : Echelle 32B+

8.1 Analyse Coût/Bénéfice par Taille de Modèle

Modèle	VRAM requise	Coût infra/mois	Performance relative	Cas d'usage
Qwen3-1.7B	4 Go	~200€ (A2)	Baseline	POC, edge
Qwen3-7B	16 Go	~500€ (A10)	+40%	Pilote, petit hôpital
Qwen3-32B	64 Go	~2 500€ (A100)	+85%	Production, CHU
Qwen3-72B	4×80 Go	~5 000€ (4×A100)	+120%	Réseau hospitalier

8.2 Feuille de Route : Passage à l'Échelle

Phase 1 : POC (ACTUEL)	Phase 2 : Pilote	Phase 3 : Production
Qwen3-1.7B + LoRA	Qwen3-7B + LoRA r=32	Qwen3-32B + LoRA r=64
GPU A2 16 Go	GPU A10 24 Go	GPU A100 80 Go
1 hopital (CHSA)	3 hopitaux pilotes	Reseau regional
Latence ~0.35s (benchmark)	Latence < 100ms	Latence < 50ms
+----- 2 mois -----+	+----- 4 mois -----+	+----- 6 mois -----+

8.3 Recommandations par Phase

Phase POC (actuelle) :

- Valider la qualité clinique sur 100 cas réels annotés
- Mesurer l'acceptabilité par les équipes soignantes
- Itérer sur les prompt templates de triage

Phase Pilote :

- Migration vers Qwen3-7B pour meilleure compréhension du français médical
- Déploiement sur GPU A10 avec optimisation mémoire (LoRA r=32, 4-bit)
- Intégration au SIH (Système d'Information Hospitalier)
- Formation des utilisateurs et procédure d'escalade

Phase Production :

- Qwen3-32B avec LoRA r=64 et quantification 4-bit GPTQ
- Déploiement multi-GPU avec Tensor Parallelism
- Redondance (2+ instances) pour haute disponibilité
- certification DM (Dispositif Médical) selon règlement UE 2017/745

9. Conclusion

- **Performance** : 30% d'exactitude de triage (100 cas), 100% de reponses sécurisées
- **Infrastructure** : vLLM operationnel, benchmark 21x, reel ~1.7x (GPU partage)
- **Modele merged** : 3.4 Go, contourne l'absence de compilation CUDA pour LoRA natif vLLM
- **Coût** : entrainement complet ~0.50€ d electricite

Note : Les contraintes choisies pour ce POC (GPU unique 16 Go, auto-hebergement Kubernetes OKD) ont ajouté une complexité externe au projet : compatibilité vLLM/CUDA, communication entre namespaces, temps de calcul allongés. Ces contraintes ont réduit le temps disponible pour l'itération sur le fine-tuning, ce qui explique en partie les résultats mitigés (30% accuracy, 53% alignment).

Prochaines étapes

1. Contrainte du format de sortie (post-traitement, template)
2. Correction du biais de prédiction (sur-triage systématique)
3. Dataset DPO ciblé triage medical (pas UltraMedical générique)
4. Validation sur dossiers CHSA réels